SDDNet: Real-Time Crack Segmentation

Wooram Choi and Young-Jin Cha

***Abstract*—This article reports the development of a pure deep learning method for segmenting concrete cracks in images. The objectives are to achieve the real-time performance while effectively negating a wide range of various complex backgrounds and crack-like features. To achieve the goals, an original convolutional neural network is proposed. The model consists of standard convolutions, densely connected separable convolution modules, a modified atrous spatial pyramid pooling module, and a decoder module. The semantic damage detection network (SDDNet) is trained on a manually created crack dataset, and the trained network records the mean intersection-over-union of 0.846 on the test set. Each test image is analyzed, and the representative segmentation results are presented. The results show that the SDDNet segments cracks effectively unless the features are too faint. The proposed model is also compared with the most recent models, which show that it returns better evaluation metrics even though its number of parameters is 88 times less than in the compared models. In addition, the model processes in real-time (36 FPS) images at 1025 × 512 pixels, which is 46 times faster than in a recent work.**

***Index Terms*—Crack segmentation, deep learning (DL), real time, separable convolution, structural health monitoring (SHM).**

I. Introduction

IN THE research on the automated assessments of infrastructures, methods based on implementing computer vision algorithms have been suggested as a potential way to replenish human visual inspections. However, computer vision algorithms are limited because they are highly susceptible to changes in environmental conditions. Moreover, they cannot be implemented in practical field usage due to the incapability of extracting sensitive features [1].

In recent years, convolutional neural networks (CNN) have been implemented instead of computer vision algorithms to overcome these limitations. CNNs are often referred to as deep learning (DL) because they are composed of deep layers of operations, in which the filters of the layers learn features automatically and hierarchically from raw data [2], [3]. Subsequently, extremely fast computations on graphics processing units (GPUs) [4] were a breakthrough in DL research, and since then the implementation of GPUs has become standardized. In the last few years, the composition of CNN architectures has been studied [5], [6], [7], [8] to achieve the better performance while reducing computational costs. As a result, many researchers have shared their pretrained models, which can be adopted directly in DL tasks (e.g., classification, object detection, and segmentation) using fine-tuning and transfer learning [9].

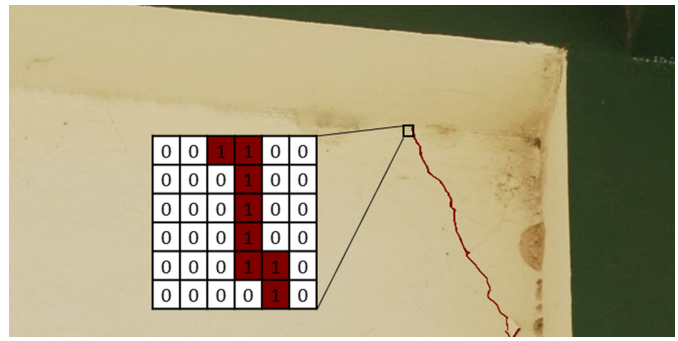

Fig. 1. Segmentation example.

The authors in [1], [10], and [11] explored the potential of DL in structural health monitoring (SHM). The authors showed that DL methods are capable of automatic feature extraction and that this potential could lead to an industrial-level application for classifying each small region of images to crack or background using of a sliding window technique. Subsequently, this approach was combined with an autonomous drone system [12]. However, in these previous methods, damage localization relied on the size of the sliding windows rather than the size of objects. Therefore, flexible localization methods were required, and faster R-CNN [13] was adopted in the SHM research on localizing damage [14], [15], [16]. However, thin, elongated, and atypical features (e.g., cracks) residing across image spaces, especially in diagonal directions, result in obfuscated localization. Therefore, an additional process is required for the quantification (i.e., measuring thickness or length) of the detected cracks. To address the abovementioned difficulties and drawbacks, we consider damage detection as pixel level segmentation rather than a problem of classification or object detection.

Object segmentation is a key DL task that aims to obtain the complete understanding of a scene in images. Object segmentation has been successfully implemented in medical image analysis [17], self-driving vehicles [18], and so on. As shown in Fig. 1, segmentation is similar to image classification except each pixel is classified rather than the entire image, in which the

Manuscript received March 20, 2019; revised July 29, 2019; accepted September 11, 2019. Date of publication October 8, 2019; date of current version April 30, 2020. The work was supported by the Natural Sciences and Engineering Research Council (NSERC) Discovery grant (Common Personal Identifier: 1262624) and Engage grant (Application 533690-18). *(Corresponding author: Young-Jin Cha.)*

The authors are with the Civil Engineering Department, the University of Manitoba, Winnipeg, MB R3T 2N2, Canada (e-mail: choiw@myumanitoba.ca; young.cha@umanitoba.ca).

Color versions of one or more of the figures in this article are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TIE.2019.2945265

0278-0046 © 2019 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on April 07,2026 at 06:01:31 UTC from IEEE Xplore. Restrictions apply.

binary numbers "1" and "0" as pixel values represent a crack pixel and a background pixel, respectively.

Crack segmentation methods that implement DL have been developed in the recent research [19], [20], [21], [22], [23]. However, these previous methods segment cracks only on monotonous backgrounds, and the trained model might not be adapted to find cracks in images with complex backgrounds. Hence, such applications are valid only if the images include surfaces without complex features or geometry, which is unlikely the case in real applications. A hybrid segmentation method for measuring the volume of concrete spalling with complex backgrounds using the combination of a faster R-CNN and a depth camera was proposed by [24]. However, it has not been successfully applied in crack quantification, because of the low resolution of the applied depth camera. Hence, segmenting cracks in 2-D on digital images remains a critical method.

In this article, cracks were selected as the objects to be segmented because they are the one of most common types of damage in civil infrastructures. Compared with other types of damage, it is harder to obtain finely grained features because cracks have subtle features that are often not visible to the naked eye. The objectives of this article are as follows. 1) build a real-time segmentation method via pure DL algorithms. 2) Negate a wide range of background features and crack-like patterns.

The rest of this article is organized as follows. Section II provides an overview of the proposed DL architecture for damage segmentation, including its general characteristics and detailed explanations of the core of the proposed model. Section III describes why and how a customized dataset is created. Section IV explains detailed training procedures and strategies. Section V illustrates and discusses the results of the proposed model. Section VI concludes this article.

II. Semantic Damage Detection Network (SDDNet)

Well-known DL models, such as ResNet [6] and DenseNet [8], were developed to recognize a large number of classes, such as the 1k classes in ImageNet [25]. On the contrary, SHM researchers deal with small numbers of classes. Therefore, we claim that a task- and domain-specific CNN architecture should be independently developed, and there would be a significant improvement in computational efficiency without performance deterioration. For these reasons, an original CNN network, which is a SDDNet that was inspired by a DenseNet and DeepLabV3+ [26], is proposed for the specific purpose of SHM.

The proposed new CNN architecture is an encoder–decoder [27] composition. Fig. 2 depicts the schematic architecture of SDDNet, where all the operations before the Decoder module functions as the encoder. In the figure, the first two operations are standard convolutions ("Conv" in Fig. 2), and the rest are newly developed or modified modules. The DenSep modules are composed of separable convolutions [28], each of which is the successive operation of a point-wise (PW) and a depth-wise (DW) convolution (see Section II-A). Each standard convolution involves a single operation, while each module involves several operations. Therefore, the parentheses and brackets shown in

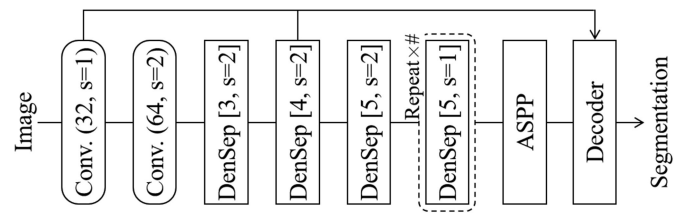

Fig. 2. Schematic SDDNet architecture.

Fig. 2 represent different contents. The first values in the parentheses in addition to the standard convolutions are the number of filters. The first values in the brackets indicate the number of separable convolutions that are involved in the corresponding modules. The second values in both the parentheses and brackets are the output stride ("s" in Fig. 2) of each operation or module, in which "s = 2" implies that the filters shift two pixels in the calculation.

Applying the output stride of 2 four times from the second "Conv. (64, s = 2)" layer to "DenSep [5, s = 2]," as shown in Fig. 2, results in reducing the spatial dimension by 1/16. Reducing dimensions is essential in any DL task to ensure computational efficiency and save GPU memory. Then, SDDNet simply repeats the DenSep module several times with the output stride of 1 to extract high-level features before multiscale-feature extraction by atrous spatial pyramid pooling (ASPP) (see Section II-B), which was modified from the original ASPP [26]. The spatial dimension of multiscale features is much smaller than that of the input image. Hence, the decoder module (see Section II-C) was designed for restoring the spatial dimension by the concatenation of low-level features at the end of the proposed network.

In this article, the filter sizes of all PW convolutions are 1 × 1, and otherwise 3 × 3. The batch normalization [29] layers follow all the convolution operations. The standard and DW convolutions involve zero paddings to retain the dimensions of the feature map (i.e., outputs of convolutions). Regarding the activation function, the rectified linear unit (ReLU) [30] is commonly used. The two standard convolutions at the beginning of the proposed network are followed by ReLUs, and the other ReLUs in DenSep, ASPP, and decoder modules are specified in Sections II-A–II-C, respectively. Several shortcuts exist in the SDDNet, which resemble those in the DenseNet architecture [8]. In the rest of this section, the architecture configurations will be explained rather than the detailed procedures of each operation, which can be found from [31].

A. DenSep Module

The DenSep module, which involves separable convolutions, is the core of the SDDNet. An example of the DenSep module with four separable convolutions is depicted in Fig. 3. In separable convolutions, calculations are performed much faster compared to the standard convolution by sequentially applying DW and PW filters [7], [28], [26]. However, all separable convolutions in SDDNet, including the DenSep, ASPP, and decoder modules, are applied in reverse order to achieve even faster

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on April 07,2026 at 06:01:31 UTC from IEEE Xplore. Restrictions apply.

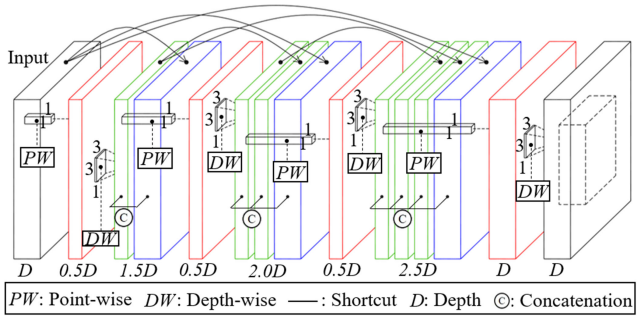

Fig. 3. DenSep module involving four separable convolutions.

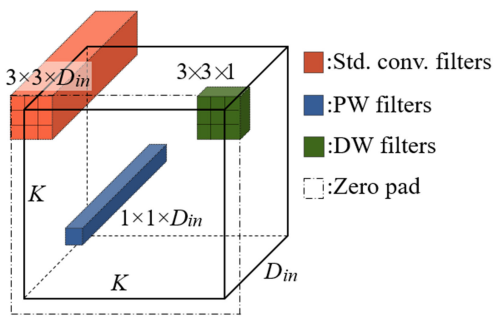

Fig. 4. Comparison of convolution operations.

TABLE I
NUMBER OF COMPUTATIONS

No. of conv.	DenSep Oper.	DenSep Mults.	Standard conv. Oper.	Standard conv. Mults.
1	PW	$0.5K^2D^2$	PW	$0.5K^2D^2$
	DW	$4.5K^2D$	Conv.	$2.25K^2D^2$
2	PW	$0.75K^2D^2$	PW	$0.75K^2D^2$
	DW	$4.5K^2D$	Conv.	$2.25K^2D^2$
3	PW	K^2D^2	PW	K^2D^2
	DW	$4.5K^2D$	Conv.	$2.25K^2D^2$
4	PW	$1.25K^2D^2$	PW	$1.25K^2D^2$
	DW	$4.5K^2D$	Conv.	$2.25K^2D^2$
5	PW	$3.0K^2D^2$	PW	$3.0K^2D^2$
	DW	$4.5K^2D$	Conv.	$9.0K^2D^2$
Total	$K^2D(6.5D+27)$		$K^2D(24.5D)$	

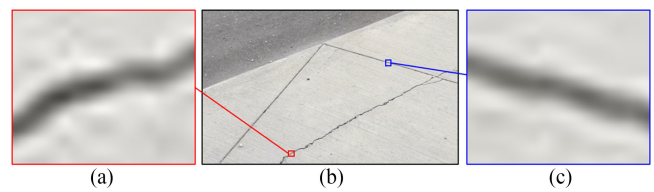

Fig. 5. Real crack versus a crack-like feature.

calculations, as shown in Fig. 3. All the PW convolutions function in reducing the depth of the inputs to dramatically reduce the computational cost [6]. The outputs of each separable convolution are designed to concatenate to all the input feature maps of later separable convolutions within each DenSep module. The last separable convolutions in the first three DenSep modules are selectively accompanied by the stride of 2 to reduce the spatial dimensions of the feature maps by 1/16 (see Section II). For simplicity, ReLUs are not shown in the figure, but they are applied after all the DW filters.

The comparison of standard, PW, and DW convolutions is shown in Fig. 4. Note that the 1×1 convolution is technically equal to the PW convolution of a separable convolution. All the convolution operations involve multiplications, and the number of multiplications can be considered the number of computations. The computational cost of each operation that is applied to the feature map having $K \times K \times D_{\text{in}}$ dimensions can be calculated by (1)–(3) for standard, PW, and DW convolutions, respectively. Note that D_{in} and D_{out} in the equations indicate the input and output depth sizes of the feature map, respectively. Separable convolution in its reverse order, which is utilized in this article, is defined by the successive operations of PW and DW convolution. Therefore, the computational cost of a separable convolution can be obtained by summing

$$9 \times K^2 \times D_{\text{in}} \times D_{\text{out}} \tag{1}$$

$$K^2 \times D_{\text{in}} \times D_{\text{out}} \tag{2}$$

$$9 \times K^2 \times D_{\text{out}}. \tag{3}$$

Based on the abovementioned equations, the number of multiplications associated with the DenSep module and a module with standard convolutions is calculated as shown in Table I. Comparing the sum of the multiplications of the modules, the DenSep module reduces approximately 70% of the computational cost compared with the standard convolution module. For example, if the size of the D is 32, the number of calculations is reduced to approximately 30% [i.e., $235 = 6.5D + 27$ and $784 = 24.5D$ (see Table I)].

B. ASPP Module

If a CNN model of segmentation is designed to deal with objects that have irregular shapes (e.g., cracks), the CNN model may require a wide field of view (FOV) rather than focusing only on a small region. The reason is that such objects likely have a small number of features, which leads to the need to gather peripheral information to ensure correct classifications. For example, as shown in Fig. 5, the features in each magnified region (i.e., a small FOV) cannot be clearly classified as cracks [see Fig. 5(a)] or crack-like features [see Fig. 5(c)] without the surrounding information of the original image (see Fig. 5). One way to collect the surrounding information is to increase the size of the filters, which, however, inevitably leads to an exponential increase in the computational cost. To deal with this drawback, a modified ASPP module is adopted in the SDDNet. The original ASPP was proposed in DeepLabV3+.

The original ASPP is composed of global average pooling (GAP), PW convolution, and three atrous separable convolutions (ASC) in the standard order (i.e., DW is followed by PW), where the only difference from separable convolution is the use of a dilated DW filter (see Fig. 6). In this article, the original ASPP is modified as follows: the PW convolution is kept, ASC

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on April 07,2026 at 06:01:31 UTC from IEEE Xplore. Restrictions apply.

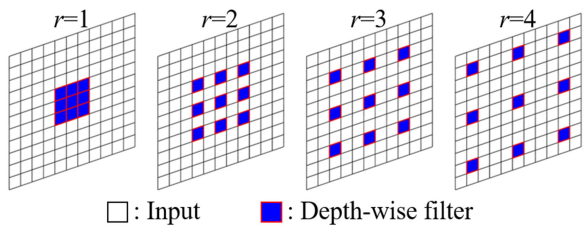

Fig. 6. Dilated DW convolution.

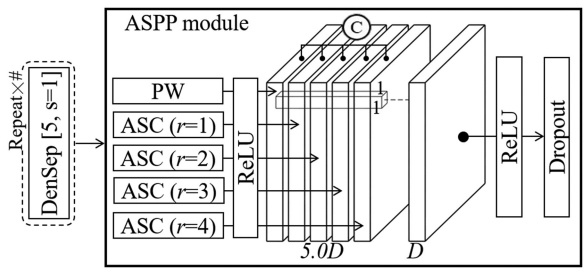

Fig. 7. Modified ASPP module.

is modified from the standard order to inverse order to reduce the computational cost, and GAP is removed because it strongly regularizes networks, whereas the SDDNet has a considerably small number of parameters that result in the performance deterioration. The dilated DW filters [32] are shown in Fig. 6, where the dilation rate (denoted as "r") implies the number of skips to the neighboring elements of a filter. The ASC with the larger dilation rate can be interpreted as the filters capturing features through a larger FOV while keeping the number of parameters of the filters.

As shown in Fig. 7, the modified ASPP module simply concatenates "ⓒ" each output of the PWs and ASCs, and it applies PW convolution to represent the concatenated feature map. Hence, the concatenation of ASCs at various dilation rates extracts multiscale features without the additional computation of large FOVs. All dilated DW convolutions in the ASPP module consist of 3×3 filters, which have the dilation rates of 1–4, respectively. To prevent overfitting, dropout [33] is used at the end of the ASPP module in training. Regarding ReLU, all the ASCs and PWs are followed by ReLUs. The effectiveness of the modified ASPP module briefly is discussed in Section V-A.

C. Decoder Module

The decoder module is designed to generate fine-grained segmentation. The module involves two-step restorations, as shown in Fig. 8. In the first step, the input feature map of the decoder module, in which the spatial dimension is 1/16 of the input images, is up-sampled by the scale factor of "4." It is concatenated with the low-level feature map from the second DenSep module (see Figs. 2 and 8). The concatenated feature map is now passed through two separate convolutions and then up-sampled again by the scale factor of "4." Thereafter, the feature map is concatenated with another low-level feature map from the first standard convolution (see Figs. 2 and 8), which has the same spatial dimensions as the input image. Finally, a 3×3 convolution is applied to the last concatenated feature map to generate the segmentation ("Seg." in Fig. 8) results. In this module, ReLUs are used after all PW and DW convolutions. The mean intersection-over-union (mIoU) loss after the softmax function is used to compute loss ("Loss cal." in the figure) and backpropagate [3] the errors in training. The loss function is discussed in Section IV-B.

D. Model Customization

The SDDNet can be customized by setting different depth values (D) and the number of repetitions of the DenSep module (see Section II). Hereafter, each customized model is denoted by SDD-R#D#, where R# and D# indicate the number of repetitions of DenSep module and the D value, respectively. The SDD-R6D64 was considered as our final model, but the SDD-R6D32 is also discussed in Section V-C. Note that the number of PW filters in the DenSep module with six repetitions was set at D instead of $0.5D$ (see Fig. 3) for the customized models reported in this article.

III. Dataset Generation

The proposed SDDNet is trained using supervised learning, which requires several data to be fed into the model. However, no public dataset suited our specific purpose. Hence, a dataset was created manually. The dataset consists of 200 digital images with various pixel resolutions. The images were collected from either the Internet or taken manually: 55 images were manually taken using smartphones. 104 images were collected from Datacenterhub [34]. A total number of 41 images were downloaded from Google image. Accordingly, each image was taken under different conditions (e.g., various distance, light intensity, FOV, and image quality). The spatial dimensions of the images in any axes are between 513 and 1920 pixels. The minimum size of the images is 513×513, and the maximum size of the images is approximately the size of high definition images (i.e., 1920×1080).

The ground truth of each image was obtained by manually selecting pixels that belonged to cracks, in which the selected pixels were labeled 1 (crack), and otherwise 0 (background). Image labeling was conducted using Affinity Photo (can be found online[1]), a commercial photo editing application. Consequently, the total number of pixels labeled as 1 was about 260 times less than the number of pixels labeled 0 (examples of the images are shown in Section V-B with the corresponding results), and any CNN model likely faces the challenge of being optimized. The dataset was divided into two subsets: 160 images for training and 40 images for testing. The cracks labeled in our dataset could be categorized according to the characteristics of crack features as follows.

1) Thick crack: a crack with a thickness of 5 or more pixels.
2) Thin crack: a crack with a thickness of 1–4 pixels.
3) Blurry crack: either a thick (i.e., thick-blurry) or thin (i.e., thin-blurry) crack with blurry features, which are still well recognizable.

[1][Online]. Available: https://affinity.serif.com/en-gb/photo

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on April 07,2026 at 06:01:31 UTC from IEEE Xplore. Restrictions apply.

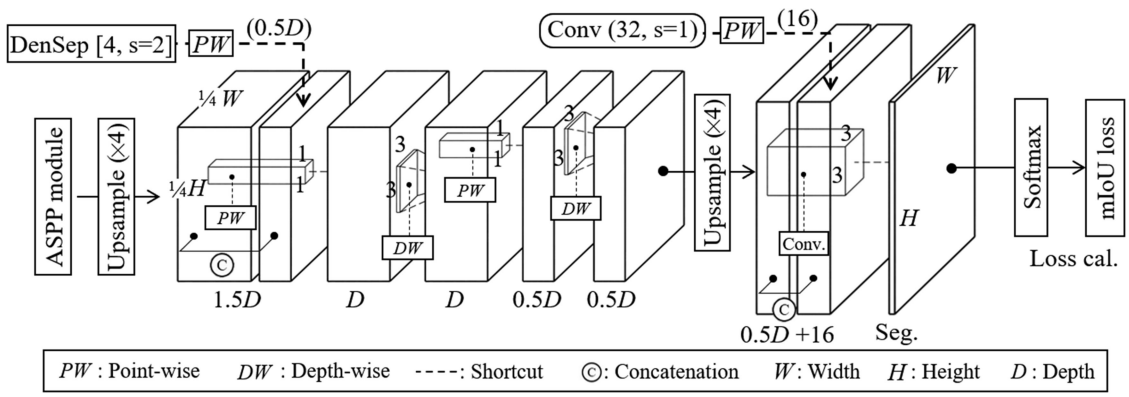

Fig. 8. Details of decoder module.

4) Faint crack: a crack that is not recognizable without careful observations.
5) Crack-like feature: a feature which resembles a crack without peripheral information (see Section II-B)

However, more detailed characteristics, such as the number of cracks and the measure of faintness were not calculated because there were no existing methods of calculating them. Sample labels can be found online.[2] Hereafter, the created dataset is referred to as the Crack200 dataset.

TABLE II
HYPERPARAMETERS FOR TRAINING SDD-R6D64

	Cityscape dataset	Crack200 dataset
lr_1/ lr_2/ *step*	0.003 / 0.00001 / 2000	0.001 / 0.00001 / 2000
Input size	513×513 (1009×1009[a])	513×513 (1009×1009[a])
Batch size	32(8[a])	32(8[a])

a: These values indicate the changed input size after the convergence of each model (see Section IV-A).

IV. TRAINING DETAILS

In this section, the details of how the SDD-R6D64 (see Section II-D) is trained are explained. First, the model was pretrained on the Cityscape dataset [35] after some modifications (see Section IV-A), followed by additional training on the Crack200 dataset. The SDDNet was coded in the Python programming language using the Tensorflow DL framework [36]. The SDDNet was deployed to a workstation, and the technical specifications were as follows:

1) CPU: Intel Core i7-6850 K;
2) GPU: Nvidia Geforce Titan Xp × 4ea;
3) RAM: 128 GB.

Note that the abovementioned GPUs were used only in training the SDDNet models. In testing, we used an old GPU, Nvidia Titan X, to highlight the real-time performance (see Section V-C).

A. Training Strategy

The input size was set to 513 × 513 pixels, which were randomly cropped from the raw training images. This specific cropping size was chosen for the following reasons: we experimentally found that sizes larger than the 1/4 cropping size of all the original images gave the best results. The reason is most likely that the cropped images contained enough contextual information. In addition, the odd number 513 was arithmetically beneficial when convolutions with strides were applied. When the model was converged, it was further trained on a larger input size. This additional training was configured to tune the dilated DW filters in the ASPP module; thereby, in testing the trained model was better on large images. When the training was completed, the trained model could accept any image size because the proposed architecture follows a fully convolutional manner [37]. The training strategy explained here was applied in both pretraining on the modified Cityscape dataset and fine tuning on the Crack200 dataset.

B. Loss Function and Hyperparameters

To train our models, we used mIoU loss, which was a modification of IoU loss [38]. The mIoU loss was calculated by (4) in a batch size of "n" with a "C" number of classes. In the equation, "$I(x, y)$" and "$U(x, y)$" indicate the sums of the approximated intersection and union, where "x" is the set of probabilities derived from the softmax function, and "y" is the set of one-hot-encoded true labels. The mIoU score, which is the most frequently used in segmentation tasks, can be calculated by disregarding the "1 − " term and substituting "x" for one-hot-encoded prediction from the following equation:

$$1 - \frac{1}{C}\frac{\sum_{i=1}^{n} I_i(x, y)}{\sum_{i=1}^{n} U_i(x, y)}. \tag{4}$$

Regarding the model optimization, the Adam optimizer [39] was used. Momentum, dropout rate, and weight decay were commonly set at 0.9, 0.5, and 0.00004 for training on both datasets, respectively. We scheduled the learning rates at each training iteration ("iter.") based on the cyclical learning rate policy [40] calculated by (5), where "X" is defined by (6). We empirically set "γ" at 0.99996. Other hyperameters including

[2] [Online]. Available: https://github.com/choiwpublic/SDDNet

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on April 07,2026 at 06:01:31 UTC from IEEE Xplore. Restrictions apply.

TABLE III
MODIFIED CITYSCAPE DATASET

Label	Object	# of image
1	Road, sidewalk, parking	3438
2	Building, wall, bridge	3448
3	Pole, traffic, light and sign	3448
4	Tree	3377
5	Human	2941
6	Vehicle	3329
0	Other	3475

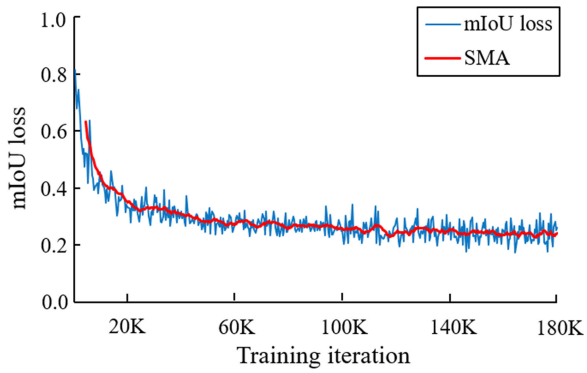

Fig. 9. mIoU loss over training iteration.

"lr_1," "lr_2," and "step" are listed in Table II

$$\mathrm{lr}_2 + (\mathrm{lr}_1 - \mathrm{lr}_2) \times \max(0, 1 - X) \times \gamma^{\mathrm{iter.}} \tag{5}$$

$$X = \left| \frac{\mathrm{iter.}}{\mathrm{step}} - 2 \times \left\lfloor \frac{1 + \mathrm{iter.}}{2 \times \mathrm{step}} \right\rfloor + 1 \right| \tag{6}$$

C. Pretraining on the Cityscape Dataset

Pretraining is a popular method because it is an effective initialization method, especially in training on a relatively small dataset [9]. Prior to training the SDD-R6D64 on the Crack200 dataset, we pretrained the model on a modified cityscape dataset. The cityscape dataset originally consisted of images of 30 different types of objects in 3475 images that were taken in various cities. We chose this dataset to verify the trainability of our model on a multiclass dataset. In addition, we considered that the image features in the cityscape dataset resembled those in our dataset. However, the proposed model is intended to deal effectively with a few classes. Therefore, we modified the original dataset by combining similar objects in the same class and converting the labels of some objects to zeros (i.e., the background). The modified cityscape dataset is shown in Table III. The SDD-R6D64 was pretrained on the modified Cityscape dataset for roughly two days, and the loss graph is given in Fig. 9. In the pretraining, the input size was changed from 513 $\times$ 513 to 1009 $\times$ 1009 at the 100 kth iteration, and it was further trained until 180 kth iteration. The pretrained models had different weights at each iteration, and there was no clear standard to define which model would be best for fine-tuning on the Crack200. We considered the models within a period of convergence to have similar contributions to fine-tuning on the Crack200. Considering the batch size of 8 and input dimension of 1009 $\times$ 1009, about 40 k iterations were enough to feed all the images of the modified Cityscape dataset into our model. The simple-moving-average (SMA in Fig. 9) of mIoU loss was, therefore, calculated with the period of 40 k iterations, as shown in Fig. 9. The models between 140 and 180 k had an SMA of 0.240 $\pm$ 0.006, and we randomly chose one of the models within the period of convergence as a pretrained model. All the pretrained filters, except the last PW filters in the decoder module (see Section II-C), were used as the initial filters in training our model on the Crack200 dataset. This technique is known as fine-tuning [9].

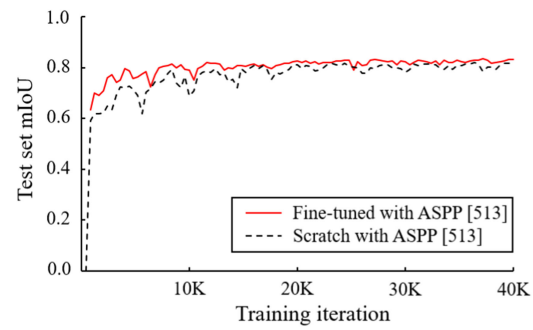

Fig. 10. Comparative profiles of SDD-R6D64 using two different training approaches.

V. RESULTS AND EXPERIMENTAL EVALUATIONS

The SDD-R6D64 was trained on the Crack200 dataset (see Section III) in addition to the training procedures (see Section IV). The trained model was tested on the Crack200's test set. The corresponding evaluation metrics and visualizations are reported in this section.

A. Learning Profiles in the Test Set

In this section, the segmentation results of the Crack200's test set are compared with different training strategies, and the corresponding discussion is provided. A loss graph over the training iterations is omitted here because the profiles from the test set included the information in the loss graph. The total duration of training on the Crack200 dataset was approximately six hours.

Fig. 10 shows the effects of the pretraining on the modified cityscape dataset. In the figure, the numbers in brackets indicate the sizes of the input images in training. "Fine-tuned" implies that the model was initialized using the filters in the pretrained model (see Section IV-C) and then trained on the crack dataset. "Scratch" indicates that the model was directly trained on the crack dataset without the adoption of a pretrained model. As the figure shows, the fine-tuned model outperformed the model trained from scratch over all training iterations.

We considered that the fine-tuning process was almost converged at 40 kth iteration with the mIoU of about 0.8, as shown in Fig. 10, and paused fine-tuning. After then, the input size was changed (see Section IV-A) from 513 $\times$ 513 to 1009 $\times$ 1009 pixels and resumed fine-tuning until 50 kth iteration.

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on April 07,2026 at 06:01:31 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
COMPARISON OF SDD-R6D64 WITH DIFFERENT TRAINING STRATEGIES

Model variation	Input size	Pre.	Re.	F1	mIoU
Fine-tuned with ASPP	1009×1009	0.805	0.834	0.819	0.846
	513×513	0.770	0.839	0.803	0.835
Fine-tuned without ASPP	1009×1009	0.752	0.855	0.800	0.833
	513×513	0.756	0.844	0.798	0.831
Scratch with ASPP	513×513	0.785	0.802	0.793	0.827

Consequently, slightly better mIoU of 0.846 were obtained as listed in Table IV, in which Precision (Pre.), recall (Re.), and F1 score are calculated by (7)–(9). In the table, the summarized results of "fine-tuned without ASPP" are also provided to check the effectiveness of the modified ASPP module.

As Table IV shows, the fine-tuned SDD-R6D64 with the modified ASPP module obtained the best overall results. Therefore, the model that achieved the mIoU of 0.846 was considered our final model. The segmentation results with comprehensive visualizations are provided in Section V-B

$$\text{Precision} = \frac{\text{True positive}}{\text{True positive} + \text{False positive}} \tag{7}$$

$$\text{Recall} = \frac{\text{True positive}}{\text{True positive} + \text{False negative}} \tag{8}$$

$$\text{F1 score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{9}$$

B. Segmentation Results

In this section, representative segmentation results are presented with comprehensive visualizations. The images shown in the figures follow the demonstration rule: the raw images and the images showing segmentation results are denoted as "Raw" and "Seg.," respectively. In the segmentation results, true positive (TP), false negative (FN), and false positive (FP) pixels are colored red, green, and blue, respectively. The combination of TP and FN pixels are interpreted as ground truths, and that of TP and FP are the pixels predicted as crack pixels by the trained model. The caption of each figure includes the corresponding mIoU values, and the size of the image is also provided in brackets beside the mIoU values. In the figures, some regions in the segmentation images are congested where the crack features are much smaller than the images, and several cracks are barely noticeable. Hence, each region is marked as a region-of-interest (ROI) and enlarged to present as fully as possible the characteristics of the proposed model.

Fig. 11 presents the segmentation results of the SDD-R6D64. The trained model successfully negated background features and detected cracks even when the tested figures contained complex crack-like features such as those shown in Fig. 11(a). In addition, the SDD-R6D64 showed the capability in segmenting very fine cracks lying on a few pixels. The common failures were observed as follows.

1) The most FN predictions were on the edges of TP pixels, but we consider such false predictions are allowable because the definitive edges of cracks cannot be obtained especially in fine cracks.
2) Faint cracks, such as those shown in Fig. 11(b)-ROI-1, Fig. 11(c)-ROI-1, and Fig. 11(c)-ROI-3, were not properly segmented (see FN pixels);
3) The cracks residing in spalling regions, as shown in Fig. 11(c)-ROI-2, were also not properly segmented (see FN pixels).

Further test results are available online.[3]

C. Comparative Studies and Discussions

In the previous sections, we made two claims: 1) building a domain- and task-specific model is much beneficial (see Section II); 2) training a model on images of monotonous backgrounds is not a proper approach (see Section I).

To support the first claim, we compared the SDDNet with DeepCrack [41], which is one of the latest CNN for crack segmentation. Liu *et al.* [41] reported the performances of six customized DeepCrack models and compared with other segmentation models. The results showed that DeepCrack models outperformed the other segmentation models, and each DeepCrack model had different merits at precision, recall, F1, and mIoU. Among the DeepCrack models, the top three were compared with a customized SDDNet model. The details of each customized DeepCrack model are provided in [41]. To further highlight the superiority of the SDDNet, we customized our model, the SDD-R6D32 (see Section II-D), to have a much smaller number of parameters than either the DeepCrack models or our final model (the SDD-R6D64). Note that the cores of the DeepCrack models were composed of VGG-16's layers [5], and VGG-16 was developed to classify images into 1k classes. The total number of parameters (i.e., weights) in the DeepCrack models was 14 million (M), while the SDD-R6D32 had only 0.160 M parameters. This was approximately 88 times smaller than that of the DeepCrack models. To ensure fair comparisons, the following were considered: 1) we used DeepCrack's train and test sets in training and testing, respectively; 2) the SDD-R6D32 was not pretrained because DeepCrack models were not pretrained. Because of the large number of parameters, the DeepCrack models were expected to show better results, but the SDD-R6D32 showed slightly better results in all aspects (precision, recall, F1, mIoU, and FPS) as shown in Table V. Regarding processing time, the DeepCrack-Aug processed an image of 544 × 384 in 109 ms (9.174 FPS). While the SDD-R6D32 processed the same size of images in 13.54 ms (75.816 FPS), and it was at least 8 times faster than all DeepCrack models.

To support the second claim, we individually trained the SDD-R6D64 on the DeepCrack and Crack200 datasets. The comparison results are shown in Table VI. Although the number of parameters of the SDD-R6D64 models was three times greater than that of the SDD-R6D32 (see Table V), this was still 26 times fewer than that of the DeepCrack models. Moreover, as shown in Table VI, the SDD-R6D64 also returned slightly better evaluation metrics (set of scores denoted by "a" in

[3][Online]. Available: https://github.com/choiw-public/SDDNet

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on April 07,2026 at 06:01:31 UTC from IEEE Xplore. Restrictions apply.

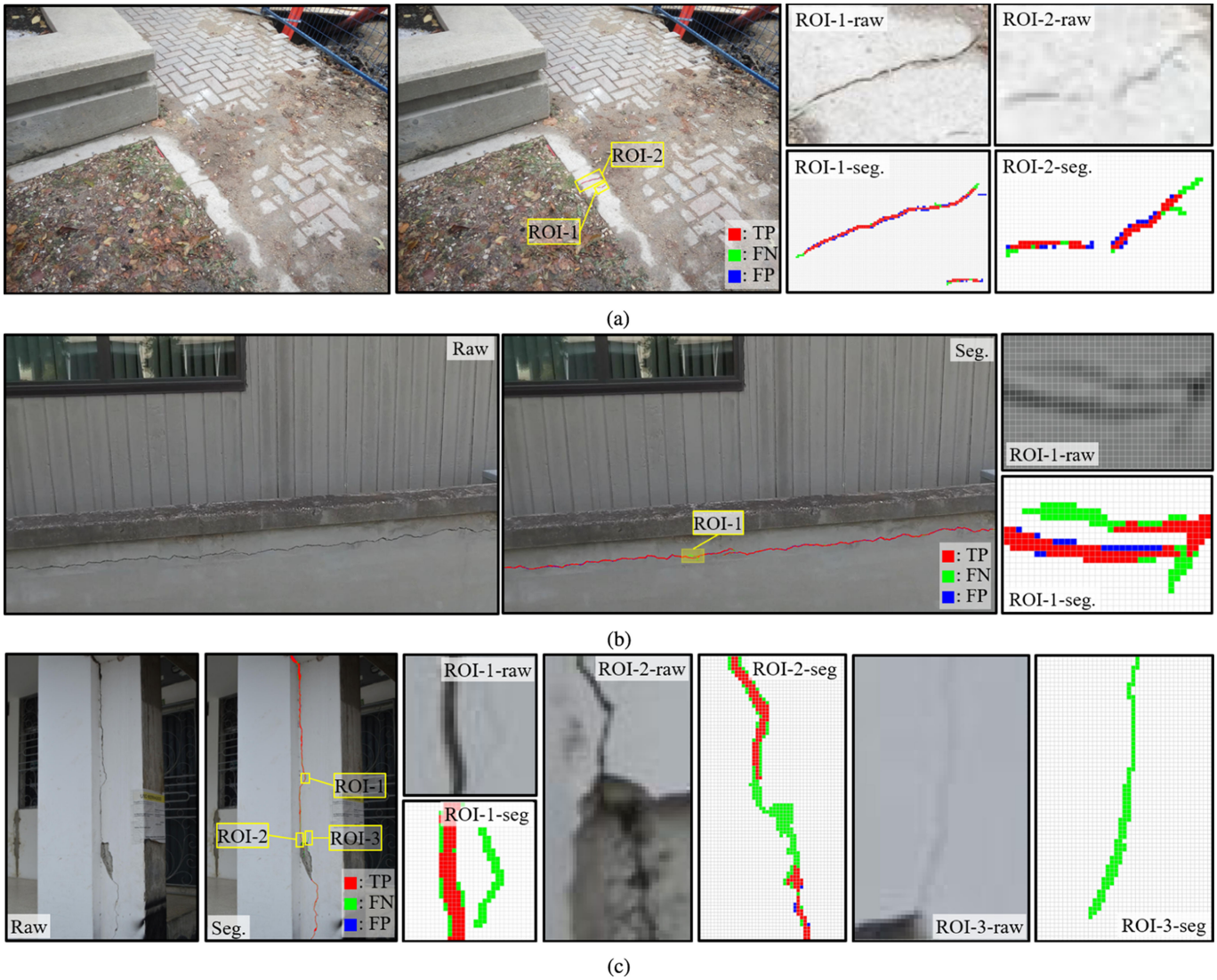

Fig. 11. Segmentation results of SDD-R6D64. (a) mIoU = 0.830 [1920 × 1440]. (b) mIoU = 0.909 [1280 × 720]. (c) mIoU = 0.898 [1276 × 1920].

TABLE V
DEEPCRACK VS. SDD-R6D32

Model variation	Par. (M)	Pre.	Re.	F1	mIoU	FPS
DeepCrack-CRF		0.868	0.846	0.857	0.836	2.500
DeepCrack-GF	14	0.852	0.866	0.859	0.859	8.475
DeepCrack-Aug		0.861	0.869	0.865	0.802	9.174
SDD-R6D32	**0.160**	**0.871**	**0.870**	**0.870**	**0.879**	**75.816**

TABLE VI
CROSS-TEST RESULTS WITH SDD-R6D64

Train set	DeepCrack's test set				Crack200's test set			
	Pre.	Re.	F1	mIoU	Pre.	Re.	F1	mIoU
DeepCrack	0.874^{a}	0.870^{a}	0.872^{a}	0.880^{a}	0.213^{b}	0.561^{b}	0.309^{b}	0.587^{b}
Crack200	0.875^{c}	0.870^{c}	0.872^{c}	0.881^{c}	0.805^{d}	0.834^{d}	0.819^{d}	0.846^{d}

a: SDD trained and tested on the DeepCrack's data.
b: SDD trained on DeepCrack's data; tested on our test data.
c: SDD trained on our data; tested on the DeepCrack's test data.
d: SDD trained and tested on the Crack200 (see Section II-D)

Table VI) than any of the DeepCrack models (see Table V). Note that the processing time of the SDD-R6D64 was recorded at 66.628 FPS, which was still at least seven times faster than all DeepCrack models. The model (SDD-R6D64) that was trained on DeepCrack's training set (i.e., monotonous backgrounds) returned worse results (set of scores denoted by "b" in Table VI) for testing the images of complex background (i.e., Crack200's test set), while the models trained on Crack200's training set recorded consistent metrics (set of scores denoted by "c" and "d" in Table VI) in both testing sets of the Crack200 and DeepCrack. These results indicated that training a model on images of monotonous backgrounds is unlikely valid in real practice. In addition, the SDD-R6D64 trained only on Crack200's training set also returned better results (set of scores denoted by "c") than all the DeepCrack models in Table V although none of DeepCrack's training images were used. The significant decrease in the precision scores from "0.874^{a}" to "0.231^{b}" revealed that the model trained on images of monotonous background resulted in extremely high FP predictions [see (7)]. This result was a

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on April 07,2026 at 06:01:31 UTC from IEEE Xplore. Restrictions apply.

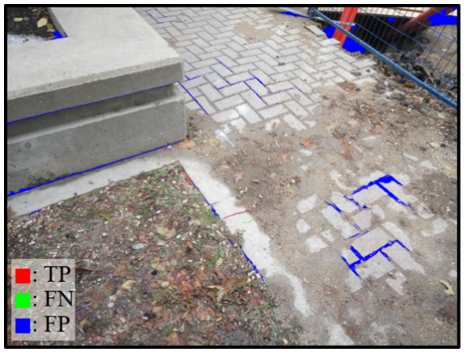

Fig. 12. Segmentation result of SDD-R6D64 trained on images of monotonous background.

TABLE VII
PROCESSING TIME OF SDD-R6D64

Image size	Milliseconds/image (FPS)
1920×1080	80.40 (12)
1025×512	27.50 (36)
720×480	18.10 (55)

clear indication of susceptibility in negating crack-like features and complex backgrounds using a model trained on images of monotonous background. In contrast, the model trained on the Crack200, retained high precision scores (0.875[c] and 0.805[d]), which implied that our final model effectively negated crack-like features and complex backgrounds. An intuitive test result of the abovementioned observations is presented in Fig. 12, which contrasts the results shown in Fig. 11(a).

To verify our model's real-time performance on large images, three different sizes of images (1920 × 1080, 1025 × 512, and 720 × 480) were tested, as shown in Table VII. As the table shows, the proposed model processed HD images in 80.40 ms (12 FPS). In addition, the images smaller than 1025 × 512 pixels were processed in real time, which is a remarkable improvement compared to the results of previous works. For example, Zhang *et al.* [21] proposed the original CNN network, CrackNet-II, and this model processed images of 1025 × 512 pixels in 1260 ms. However, our final model processed the same size of images in 27.50 ms (36 FPS), which was 46 times faster than in CrackNet-II.

Despite the merits of SDDNet discussed in this section, the model performance was affected due to poor quality of crack features [incapability of segmenting faint cracks (see Section V-B)]. Hence, certain applications (e.g., asphalt and railway inspections in real time) may require a camera that can capture fine features at high speeds, which, however, is a common challenge in any DL models used to segment objects.

In addition to the performances of SDDNet, our model shows the potential for effective use in real practice. One major concern in deploying a model to practical devices, such as drones, wearable devices, etc., is storage. The SDD-R6D64 required less than 3 megabytes (MB), which would be beneficial in configuring hardware, reducing physical dimensions, and lowering technical requirements as well as cost savings in building SHM equipment.

VI. CONCLUSION

In this article, the new DL architecture, the SDDNet-V1, was proposed for the real-time segmentation of superficial cracks in structures. The SDDNet was composed of standard convolutions, several DenSep modules, a modified ASPP module, and a decoder module. One of the objectives of this article was to segment cracks in images of a wide range of background features. However, no public dataset coincided with this objective at the time of research. Hence, the Crack200 dataset was manually created. The SDDNet was pretrained on the modified cityscape dataset and then trained on the Crack200 dataset to improve its performance. The trained model was tested on the test set of the Crack200 dataset, and several images of the results were presented and discussed. This article was conducted to evaluate the proposed model's performance by comparing it with recent models. The results showed that building a domain- and task-specific model would be beneficial and that training a model on images of monotonous backgrounds would not be valid in real practice. The proposed model is expected to be one of the best options in crack segmentation based on the following results.

1) The SDDNet was able to effectively negate complex backgrounds and crack-like features.
2) The SDDNet successfully segmented cracks unless the crack features were too faint. Accordingly, the SDDNet recorded an mIoU of 0.846 and an F1 score of 0.819.
3) The SDDNet showed the better performance in all aspects compared to recent developments even though the model size was 88 times smaller than the compared models.
4) The SDDNet showed the real-time performance (36 FPS) on images of 1025 × 512 pixels, which was 46 times faster than that in a recent development.
5) The SDDNet required less than 3 MB of storage in a device, which could lead to flexible choices in configuring hardware.

The common challenge for the susceptibility to image quality was not addressed in this article because it is related to a camera's specifications than a DL model. This challenge will be investigated in future research.

REFERENCES

[1] Y.-J. Cha, W. Choi, and O. Büyüköztürk, "Deep learning-based crack damage detection using convolutional neural networks," *Comput.-Aided Civil Infrastructure Eng.*, vol. 32, no. 5, pp. 361–378, May 2017, doi: 10.1111/mice.12263.
[2] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
[3] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016. [Online]. Available: http://www.deeplearningbook.org.
[4] D. Steinkraus, I. Buck, and P. Simard, "Using gpus for machine learning algorithms," in *Proc. IEEE Int. Conf. Document Anal. Recognit.*, 2005, pp. 1115–1120, doi: 10.1109/ICDAR.2005.251.
[5] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.
[6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on April 07,2026 at 06:01:31 UTC from IEEE Xplore. Restrictions apply.

[7] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 1251–1258.

[8] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 4700–4708.

[9] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" in *Proc. Adv. Neural Inf. Process. Syst.*, 2014, pp. 3320–3328.

[10] B. Li, K. C. Wang, A. Zhang, E. Yang, and G. Wang, "Automatic classification of pavement crack using deep convolutional neural network," *Int. J. Pavement Eng.*, pp. 1–7, 2018.

[11] N. T. H. Nguyen, T. H. Le, S. Perry, and T. T. Nguyen, "Pavement crack detection using convolutional neural network," in *Proc. 9th Int. Symp. Inf. Commun. Technol.*, 2018, pp. 251–256.

[12] D. Kang and Y.-J. Cha, "Autonomous UAVs for structural health monitoring using deep learning and an ultrasonic beacon system with geotagging," *Comput.-Aided Civil Infrastructure Eng.*, vol. 33, no. 10, pp. 885–902, Oct. 2018, doi: 10.1111/mice.12375.

[13] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 91–99.

[14] Y.-J. Cha, W. Choi, G. Suh, S. Mahmoudkhani, and O. Büyüköztürk, "Autonomous structural visual inspection using region-based deep learning for detecting multiple damage types," *Comput.-Aided Civil Infrastructure Eng.*, vol. 33, no. 9, pp. 731–747, Sep. 2018, doi: 10.1111/mice.12334.

[15] H. Maeda, Y. Sekimoto, T. Seto, T. Kashiyama, and H. Omata, "Road damage detection and classification using deep neural networks with smartphone images," *Comput.-Aided Civil Infrastructure Eng.*, vol. 33, no. 12, pp. 1127–1141, Dec. 2018, doi: 10.1111/mice.12387.

[16] Y. Xue and Y. Li, "A fast detection method via region-based fully convolutional neural networks for shield tunnel lining defects," *Comput.-Aided Civil Infrastructure Eng.*, vol. 33, no. 8, pp. 638–654, Aug. 2018, doi: 10.1111/mice.12367.

[17] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Proc. Int. Conf. Med. Image Comput. Comput.-Assisted Int.*, 2015, pp. 234–241.

[18] M. Siam, S. Elkerdawy, M. Jagersand, and S. Yogamani, "Deep semantic segmentation for automated driving: Taxonomy, roadmap and challenges," in *Proc. IEEE 20th Int. Conf. Intell. Transp. Syst.*, 2017, pp. 1–8.

[19] F. Nayyeri, L. Hou, J. Zhou, and H. Guan, "Foreground–background separation technique for crack detection," *Comput.-Aided Civil Infrastructure Eng.*, vol. 34, no. 6, pp. 457–470, 2018.

[20] X. Yang, H. Li, Y. Yu, X. Luo, T. Huang, and X. Yang, "Automatic pixel-level crack detection and measurement using fully convolutional network," *Comput.-Aided Civil Infrastructure Eng.*, vol. 33, no. 12, pp. 1090–1109, Dec. 2018, doi: 10.1111/mice.12412.

[21] A. Zhang *et al.*, "Deep learning-based fully automated pavement crack detection on 3d asphalt surfaces with an improved cracknet," *J. Comput. Civil Eng.*, vol. 32, no. 5, Sep. 2018, Art. no. 04018041, doi: 10.1061/(ASCE)CP.1943-5487.0000775.

[22] C. V. Dung *et al.*, "Autonomous concrete crack detection using deep fully convolutional neural network," *Autom. Construction*, vol. 99, pp. 52–58, Mar. 2019, doi: 10.1016/j.autcon.2018.11.028.

[23] F. Ni, J. Zhang, and Z. Chen, "Pixel-level crack delineation in images with convolutional feature fusion," *Structural Control Health Monit.*, vol. 26, no. 1, Jan. 2019, Art. no. e2286, doi: 10.1002/stc.2286.

[24] G. H. Beckman, D. Polyzois, and Y.-J. Cha, "Deep learning-based automatic volumetric damage quantification using depth camera," *Autom. Construction*, vol. 99, pp. 114–124, Mar. 2019, doi: 10.1016/j.autcon.2018.12.006.

[25] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 248–255.

[26] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, "Encoder-decoder with atrous separable convolution for semantic image segmentation," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 801–818.

[27] V. Badrinarayanan, A. Kendall, and R. Cipolla, "Segnet: A deep convolutional encoder-decoder architecture for image segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 12, pp. 2481–2495, Dec. 2017.

[28] A. G. Howard *et al.*, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*.

[29] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," 2015, *arXiv:1502.03167*.

[30] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in *Proc. 27th Int. Conf. Mach. Learn.*, 2010, pp. 807–814.

[31] V. Dumoulin and F. Visin, "A guide to convolution arithmetic for deep learning," 2016, *arXiv:1603.07285*.

[32] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, "Rethinking atrous convolution for semantic image segmentation," 2017, *arXiv:1706.05587*.

[33] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929–1958, 2014.

[34] C. Sim *et al.*, "Performance of low-rise reinforced concrete buildings in the 2016 ecuador earthquake," Sep. 2016. doi: 10.4231/R7ZC8111.

[35] M. Cordts *et al.*, "The cityscapes dataset for semantic urban scene understanding," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 3213–3223.

[36] M. Abadi *et al.*, "Tensorflow: A system for large-scale machine learning," in *Proc. 12th Symp. Oper. Syst. Des. Implementation*, 2016, pp. 265–283.

[37] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 3431–3440.

[38] M. A. Rahman and Y. Wang, "Optimizing intersection-over-union in deep neural networks for image segmentation," in *Proc. Int. Symp. Vis. Comput.*, 2016, pp. 234–244.

[39] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.

[40] L. N. Smith, "Cyclical learning rates for training neural networks," in *Proc. IEEE Winter Conf. Appl. Comput. Vis.*, 2017, pp. 464–472.

[41] Y. Liu, J. Yao, X. Lu, R. Xie, and L. Li, "Deepcrack: A deep hierarchical feature learning architecture for crack segmentation," *Neurocomputing*, vol. 338, pp. 139–153, 2019, doi: 10.1016/j.neucom.2019.01.036.

Wooram Choi received the M.Sc. degree in structural engineering from Hanyang University, Seoul, South Korea, in 2008. He has been working toward the Ph.D. degree in the Civil Engineering Department at the University of Manitoba, Winnipeg, MB, Canada, since 2015.

From 2008 to 2011, he was with Korea Consultant International, where he was a Researcher with the Research and Development Department. Currently, his works have been involved with deep-learning implemented structural health monitoring. His research interest includes the development of interdisciplinary applications with machine learning algorithms.

Young-Jin Cha was born in Jinju, South Korea. He received Ph.D. degree in civil engineering from Texas A&M University, College Station, TX, USA, in 2008.

He was a Postdoctoral Associate with the Massachusetts Institute of Technology (MIT), Cambridge, MA, USA, and with The City College of New York (CCNY), New York, NY, USA, from 2012 to 2014, and from 2009 to 2012, respectively. He is currently an Assistant Professor of the Civil Engineering Department at the University of Manitoba, Winnipeg, MB, Canada. He authored or coauthored 70 peer-reviewed articles with two international patents on these topics. His research interests include advanced deep machine learning, image processing, and automation for structural health monitoring using robotics and unmanned aerial vehicles, defect detection of foods and commercial products, and disease detection on medical images and signals.

Dr. Cha is an Advisory Editor of Engineering Reports, Wiley, and Editorial Board Member, Current Analysis on Instrumentation and Control, Mesford. He is a member of American Society of Civil Engineers.

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on April 07,2026 at 06:01:31 UTC from IEEE Xplore. Restrictions apply.